9. Gyakorlat

Python kurzus

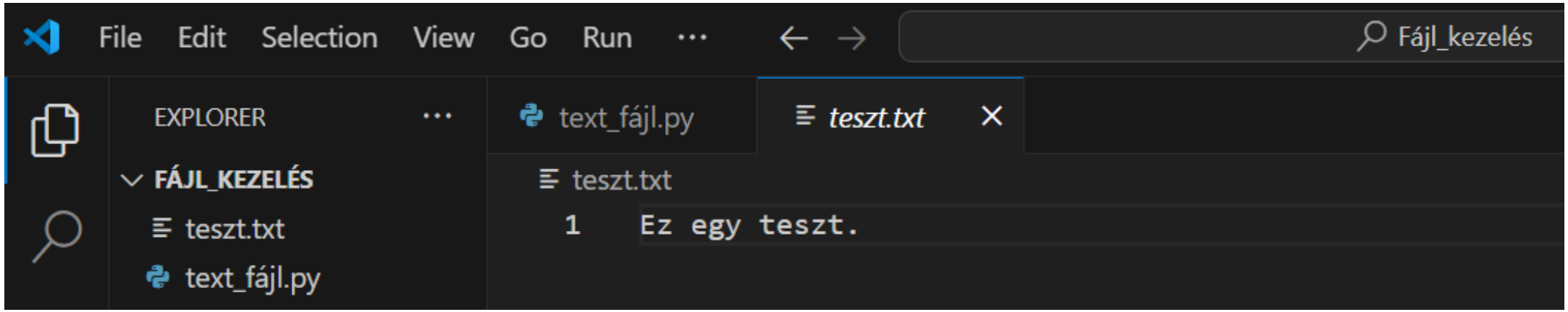
Fájlkezelés, time modul, Flask

Destruktorok – az objektumok megszüntetése

A **destruktor** metódus egy objektum megszüntetését és a memória felszabadítását eredményezi. A destruktor neve **__del__**.

A destruktor akkor hasznos, ha valamilyen erőforrást (fájlokat, hálózati kapcsolatokat) kell felszabadítani vagy bezárni, amikor egy objektum már nincs használatban.

1. példa: destruktor a text_fájl.py-ban



text_fájl.py

```
class Fájl:
    def __init__(self, név):
        self.név = név
        self.fájl = open(név, 'w')
        print(f"A(z) {self.név} fájl megnyitva írásra.")
    def írás(self, szöveg):
        self.fájl.write(szöveg)
    def __del__(self):
        self.fájl.close()
        print(f"A(z) {self.név} fájl lezárva.")

fájl = Fájl("teszt.txt")
fájl.írás("Ez egy teszt.")
#del fájl
print()
```

#del fájl esetén:

```
A(z) teszt.txt fájl megnyitva írásra.

A(z) teszt.txt fájl lezárva.
```

del fájl esetén:

```
A(z) teszt.txt fájl megnyitva írásra.
A(z) teszt.txt fájl lezárva.
```

2.példa:
A **del** függvény szerepe:

```
class BankSzámla:
   def __init__(self, tulajdonos, kezdő_egyenleg):
      self.tulajdonos = tulajdonos
      self.egyenleg = kezdő_egyenleg
      print(f"{self.tulajdonos} számlája megnyitva {self.egyenleg} egyenleggel.")
   def befizet(self, összeg):
      self.egyenleg += összeg
      print(f"{összeg} befizetve. Új egyenleg: {self.egyenleg}.")
   def kivesz(self, összeg):
      if összeg > self.egyenleg:
         print("Nincs elég fedezet.")
      else:
         self.egyenleg -= összeg
         print(f"{összeg} kivéve. Új egyenleg: {self.egyenleg}.")
   def __del__(self):
      print(f"{self.tulajdonos} számlája lezárva.")

számla = BankSzámla("Anna", 100000)
számla.befizet(30000)
számla.kivesz(5000)
#del számla
print()
```

```
Anna számlája megnyitva 100000 egyenleggel.
30000 befizetve. Új egyenleg: 130000.
5000 kivéve. Új egyenleg: 125000.

Anna számlája lezárva.
```

Fájlkezelés

A program egy absztrakt **stream**-en keresztül kommunikál a fájlokkal. A stream-en végzett műveletek után az operációs rendszer kernelében lévő mechanizmusok manipulálják a valós fájlokat. A fájl megnyitása és lezárása az **open – close**, illetve a **with** kulcsszavak alkalmazásával történik, a példákon bemutatott módon.

A stream két alapművelete: **Olvasás - Írás**

```
try:
   stream = open("C:\Users\User\Desktop\file.txt", "rt")
   ###
   stream.close()
except Exception as e:
   print(e)
```

Text mode	Description
rt	read
wt	write
at	append
r+t	read and update
w+t	write and update

Exception, ha a fájl nem található:

```
try:
   stream = open("fajl.txt", "rt")
   ###
   stream.close()
except Exception as e:
   print(e)
```

[Errno 2] No such file or directory: 'fajl.txt'

Szöveges fájl feldolgozása:

Olvasás: read() metódussal

```
stream = open("read.txt", "rt", encoding ="utf-8")
content = stream.read()
print(content)
```

```
from os import strerror
try:
    cnt =0
    st = open('text.txt', "rt")
    content =st.read()
    for ch in content:
        print(ch, end="")
        cnt +=1
    st.close()
    print(cnt)
except IOError as e:
    print("I/O error occurred: ", strerror(e.errno))
```

Fájlba írás: write() metódussal

```
from os import strerror
try:
    st = open('newtext.txt', 'wt')
    for i in range(7):
        st.write("line #" + str(i+1) +"\n")
    st.close()
except IOError as e:
    print("I/O hiba: ", strerror(e.errno))
```

newtext.txt:

```
line #1
line #2
line #3
line #4
line #5
line #6
line #7
```

Fájl megnyitása a with kulcsszóval: a with() egy kontextuskezelőt használ, amely biztosítja, hogy a fájl **automatikusan bezáródik**, amikor a blokk végrehajtása befejeződik, még akkor is, ha kivétel történik.

TXT fájl soronkénti írása és olvasása:

```
with open("kimenet.txt", "w") as fajl:
   fajl.write("Ez az 1.sor.\n")
   fajl.write("Ez a 2.sor.\n")
```

A kimenet.txt fájlt **írási módban** megnyitva, ha a fájl már létezik, annak tartalma **felülíródik**.

```
with open("adatok.txt", "r") as fajl:
   for sor in fajl:
      print(sor)
      #print(sor.strip())    # A strip() eltávolítja a felesleges sortöréseket
print('vége')
```

```
Ez az 1.sor.

Ez a 2.sor.

vége
```

strip() esetén:

```
Ez az 1.sor.
Ez a 2.sor.
vége
```

Adatok hozzáfűzése meglévő TXT fájlhoz:

```
with open("kimenet.txt", "a") as fajl:
   fajl.write("Ez egy uj sor, amit hozzaadunk.\n")
```

Az **append** módban új sorokat fűzhetünk hozzá egy már meglévő fájlhoz, a fájl tartalma **nem íródik felül.**

Stream problémák típusai:

```
from os import strerror
try:
   with open("c:/users/user/Desktop/file.txt","r") as fajl:
      content = falj.read()
      print(content)
      print(type(content))
except Exception as e:
   if e.errno == errno.ENOENT:
      print("A fájl nem létezik.")
   elif e.errno ==errno.EMFILE:
      print("Túl sok fájlt nyitott meg.")
```

```
AAAA AAA AAAAAAA AAAAAAAA A AAAAA AAA
<class 'str'>
```

Gyakori hibaüzenetek:

errno.EACCES → Permission denied
errno.EBADF → Bad file descriptor
errno.EEXIST → File exists
errno.EISDIR → Is a directory
errno.ENOENT → No such file or directory

Példa: exception-kód kiírási esetek az errno.ENOENT alkalmazásával:

```
from os import strerror
try:
   with open("file.txt","r") as s:
   content = s.read()
except Exception as e:
   print("I/O hiba keletkezett: ", e.strerror)
   print("Hiba típusa: ", type(e))      # e.__class__
   print("Hiba üzenet: ", e.__str__())
   print("Hiba kód: ", e.errno)
   print('e.args: ', e.args)
```

```
I/O hiba keletkezett:  No such file or directory
Hiba típusa:  <class 'FileNotFoundError'>
Hiba üzenet:  [Errno 2] No such file or directory: 'file.txt'
Hiba kód:  2
e.args:  (2, 'No such file or directory')
```

Az összes errno konstans kiírása

```
import errno
db = 0
for name, value in errno.errorcode.items():
   print(f"{name} → {value}")
   db += 1
print("Összesen: ", db, "db errno konstns van.")
```

```
...
2 → ENOENT
13 → EACCES
17 → EEXIST
21 → EISDIR
...
Összesen:  101 db errno konstans van.
```

A Python **os modulja** POSIX-kompatibilis (**Portable Operating System Interface szabvány**) funkciókat biztosít fájlműveletekhez és hibakezeléshez.

Az **errno modulban** általában 133 különböző szabványos hibaüzenet érhető el.

Példák az IO műveletekhez kapcsolt hibaüzenetekre:

errno.EACCES → Permission denied	**Nincs jogosultság a fájl megnyitásához.**
errno.EBADF → Bad file descriptor	**Hibás fájlleíró.**
errno.EEXIST → File exists	**A fájl már létezik.**
errno.EFBIG → File too large	**A fájl túl nagy.**
errno.EISDIR → Is a directory	**A fájl egy könyvtár.**
errno.EMFILE → Too many open files	**Túl sok fájlt nyitott meg.**
errno.ENOENT → No such file or directory	**A fájl nem létezik.**
errno.ENOSPC → No space on device	**Nincs elegendő hely a fájl létrehozásához.**
errno.ENOTDIR → Not a directory	**A fájl nem könyvtár.**
errno.ENOTEMPTY → Directory not empty	**A könyvtár nem üres.**
errno.EINVAL → Invalid argument	**Érvénytelen fájlnév.**
errno.ELOOP → Too many symbolic links encountered	**Túl sok szimbolikus hivatkozás.**
errno.EROFS → Read-only file system	**A fájl csak olvasható.**
errno.ENAMETOOLONG → File name too long	**A fájlnév túl hosszú.**

adatok.txt: AAAAAAA AAAAAAA AAAAAAA AAAAAAA AAAA AAA

```
from os import strerror
try:
   db =0
   s = open("adatok.txt", "rt")
   content =s.read()
   for c in content:
      if c == " ":
         db +=1
   s.close()
   print(db)
except IOError as e:
   print("I/O hiba keletkezett: ", strerror(e.errno))
```

```
from os import strerror
try:
   cnt =0
   s = open('adatok.txt', "rt")
   content =s.read()
   for ch in content:
      print(ch, end='')
      cnt +=1
   s.close()
   print(cnt)
except IOError as e:
   print("I/O error occurred: ", strerror(e.errno))
```

```
from os import strerror
try:
   with open('uj_text.txt', 'w')  as f:
      for i in range(10):
         f.write(str(i) + "x" + str(i+1) + " = " + str(i*(i+1)) + "\n")
except IOError as e:
   print("I/O error occurred: ", strerror(e.errno))
```

```
from os import strerror
n = int(input('Kérek egy egész számot: '))
try:
   with open('uj_tabla.txt', 'w') as f:
      for i in range(11):
         f.write(str(i) + "x" + str(n) + " = " + str(i*n) + "\n")
except IOError as e:
   print("I/O error occurred: ", strerror(e.errno))
```

1. feladat:

Prof Strict az egyetemi óráin a következő feljegyzéseket készíti egy **szöveges fájlban.**
A fájl minden sora három elemet tartalmaz, szóközzel elválasztva:
a hallgató keresztnevét, vezetéknevét, és az egyes órákon a kapott pontok számát.
Minden hallgató többször is megjelenhet a fájlban.

A prof.txt tartalma:

```
John Smith 5
Anna Boleyn 4.5
John Smith 2
Anna Boleyn 11
Andrew Cox 1.5
```

A kimeneti summary.txt:

```
Andrew Cox    1.5
Anna Boleyn   15.5
John Smith    7.0
```

```
class StudentsDataException(Exception):
    pass
class BadLine(StudentsDataException):
    pass
class FileEmpty(StudentsDataException):
    pass
```

Készítsd el a summary.txt fájlt, ami a minta szerint megadja a hallgatók összesített eredményét. A kivételkezelést a minta szerinti osztályokkal oldd meg.

CSV fájl kezelése

A **Comma-Separated Values** típusú fájlok táblázatos adatokat tárolnak, amelyek mezőit vesszővel választjuk el egymástól. Szükséges a **csv modul** importálása.

a) CSV fájl írása

A newline='' paraméter ajánlás a csv modulban, hogy üres sorok ne jelenjenek meg a fájlban.

csv.writer(fajl) létrehoz egy writer objektumot a megadott fájlobjektumra.

A writer objektum **writerows()** metódusa a listaelemeket (egy-egy sort) kiírja a fájlba.

```
import csv
adatok = [
   ["Nev", "Kor", "Varos"],
   ["Anna", 25, "Budapest"],
   ["Bela", 30, "Szeged", "\n"],
   ["Cecilia", 28, "Debrecen"]
]
with open("kimenet.csv", "w", newline='') as fajl:
   iro = csv.writer(fajl)
   iro.writerows(adatok)
```

b) CSV fájl olvasása

A **csv.reader(fajl)** egy reader objektumot ad vissza, melynek elemei a fájl sorainak megfelelő listák, amelyeket a ciklus megjelenít.

```
import csv
with open("adatok.csv", "r") as fajl:
   olvaso = csv.reader(fajl)
   print(olvaso)
   for sor in olvaso:
      print(sor)
```

```
<_csv.reader object at 0x000002C6F0473AC0>
['Nev', 'Kor', 'Varos']
['Anna', '25', 'Budapest']
['Bela', '30', 'Szeged']
['Cecilia', '28', 'Debrecen']
```

Az os modul

Az operációs rendszer elérése Pythonból:

```
import os, platform
print(os.name())
print(platform.system())
```

Al-directory létrehozása:

```
os.makedirs("first_directory/second_directory")
os.chdir("first_directory")
print(os.listdir())
```

Váltás és helyzetmeghatározás:

```
os.chdir("second_directory")
print(os.getcwd())
```

Directory létrehozása:

```
os.mkdir("first_directory")
print(os.listdir())
```

Directory törlése Pythonban:

```
os.mkdir("example_directory")
print(os.listdir())
os.rmdir("example _directory")
print(os.listdir())
```

A sys modul attribútumai:

```
import sys
print(sys.version)
print(sys.executable)
print(sys.path)
```

A bemutatott összes függvény lecserélhető a **system()** nevű függvényre, amely a neki stringként átadott parancsot végrehajtja:

```
import os
returned_value = os.system("mkdir first_directory")
print(returned_value)
```

Bevezetés a datetime modulba

Az aktuális helyi dátum lekérése és dátumobjektumok létrehozása:

```
from datetime import date
today = date.today()
print("Today:", today)
print("Year:", today.year)
print("Month:", today.month)
print("Day:", today.day)
```

Dátum objektum létrehozása ISO formátum használatával:

```
d = date.fromisoformat('2025-11-12')
print(d)
```

YYYY-MM-DD

A hét hányadik napja?

```
d = date(2025,11,12)
print(d.weekday())  # Monday=0, Sunday=6
```

Dátumobjektum létrehozása időbélyegből:

```
import time
timestamp = time.time()
print("Timestamp:", timestamp)
d = date.fromtimestamp(timestamp)
print("Date:", d)
```

time objektum létrehozása:

```
from datetime import time
t = time(14, 53, 20, 1)
print("Time:", t)
print("Hour:", t.hour)
print("Minute:", t.minute)
print("Second:", t.second)
print("Microsecond:", t.microsecond)
```

Az időbélyeg kiolvasása:

A **timestamp()** metódus egy lebegőpontos értéket ad vissza, amely a datetime objektum által jelzett dátum és idő és 1970. január 1. 00:00:00 között eltelt másodpercek számát fejezi ki:

```
from datetime import datetime
dt = datetime(2020, 10, 4, 14, 55)
print("Timestamp:", dt.timestamp())
```

A date és a time formázása:

```
from datetime import time
from datetime import datetime
t = time(14, 53)
print(t.strftime("%H:%M:%S"))
dt = datetime(2020, 11, 4, 14, 53)
print(dt.strftime("%y/%B/%d %H:%M:%S"))
```

A naptár használata:

```
import calendar
print(calendar.calendar(2024))
print(calendar.month(2020, 10))
calendar.prmonth(2024, 12)

calendar.setfirstweekday(calendar.SUNDAY)
```

Példa:

datetime.timedelta() használata két időpont közti különbség kiszámítására,

a timedelta létrehozása számokból, és az eredmény bontása nap/óra/perc/másodperc formára.

```
from datetime import datetime, timedelta

now = datetime.now()
szuletesi_datum = datetime(2005, 3, 15, 8, 30)  # 2005-03-15 08:30:00

eltelt = now - szuletesi_datum      # ez ad egy timedelta objektumot
print("Eltelt idő (alap):", eltelt)         # pl. "12976 days, 05:12:34.123456"
print("Összes nap:", eltelt.days)
print("Összes másodperc:", eltelt.total_seconds())

days = eltelt.days    # bontás nap/óra/perc/másodperc formára
secs_resten = eltelt.seconds   # napon belüli másodpercek (0..86399)
hours = secs_resten // 3600
minutes = (secs_resten % 3600) // 60
seconds = secs_resten % 60
print(f"{days} nap, {hours} óra, {minutes} perc, {seconds} másodperc")

# létrehozás konkrét értékekből
d1 = timedelta(days=7)            # egy hét
d2 = timedelta(hours=5, minutes=30)      # 5 óra 30 perc
print("Összegzett timedelta:", d1 + d2)    # 7 days, 5:30:00

jovo = now + timedelta(days=30)    # hozzáadás/kivonás dátumhoz
mult_hete = now - timedelta(weeks=2)
print("30 nap múlva:", jovo)
print("2 héttel ezelőtt:", mult_hete)
```

Flask és Jinja2

A **Flask** és a beépített **Jinja2** sablonmotor a Python alapú webfejlesztés eszközei, amelyek együttműködve lehetővé teszik **dinamikus webalkalmazások** létrehozását.

A **Flask** egy **WSGI** (Web Server Gateway Interface) kompatibilis Python webkeretrendszer. A Flask **Werkzeug könyvtára** implementálja a WSGI interfészt és biztosítja Flask Request/Response objektumok kezelését.

A WSGI egy protokoll Python webalkalmazások (pl. Flask, Django) és webkiszolgálók (pl. Gunicorn, Uvicorn) közötti együttműködésre.

A Flask fő jellemzői:

- Webalkalmazások készítése: Flask segítségével könnyen létrehozhatók **REST API**-k és **dinamikus weboldalak**.
- A Flask egy "**microframework**": az alapvető funkciókat biztosítja, és szabadon bővíthető különféle bővítményekkel.
- **Routing:** URL-eket társíthatunk Python függvényekhez, pl. az @app.route dekorátorral.
- Támogatja a **GET, POST, PUT, DELETE** HTTP metódusok kezelését.
- **JSON támogatás:** Könnyen kezelhetők JSON adatok.

Flask telepítése: `pip install flask`

Jinja2:

egy sablonmotor, amelyet a Flask használ HTML oldalak dinamikus generálására. Tartalmaz űrlapokhoz, navigációs sávokhoz, gombokhoz, táblázatokhoz és más UI-elemekhez szerkesztési lehetőségeket. Jinja2 sablonnyelvet használ.

Fő jellemzői:

- **Dinamikus HTML generálás**: Lehetővé teszi, hogy Python változókat és logikát ágyazzunk be HTML sablonokba.
- **Vezérlési szerkezetek**: Támogatja az if, for és más vezérlési szerkezeteket a sablonokban.
- **Sablonöröklés**: Segítségével új sablonok készíthetők meglévő sablonok kiterjesztésével, ami megkönnyíti az ismétlődő kód elkerülését.
- **Biztonság**: Automatikusan escape-eli a HTML kódot, hogy megvédjen az XSS támadásoktól.

Együttműködésük: a Flask a Jinja2-t használja a HTML sablonok **renderelésére**:

- A Flask egy Python függvényben adatokat készít elő.
- Az adatokat és változókat átadja egy Jinja2 sablonnak a **render_template()** függvénnyel (a sablon feldolgozása, megjelenítése).
- A Jinja2 sablon a kapott adatokat beilleszti a HTML-be, és a kész oldalt visszaküldi a böngészőnek.
- A sablonokat a **templates** mappában tároljuk, a Flask alapértelmezetten itt keresi a sablonokat.
- HTML oldalon belül {{data}} formában hivatkozhatók változók.
- {% ... %} formában megadhatók a vezérlési szerkezetek.

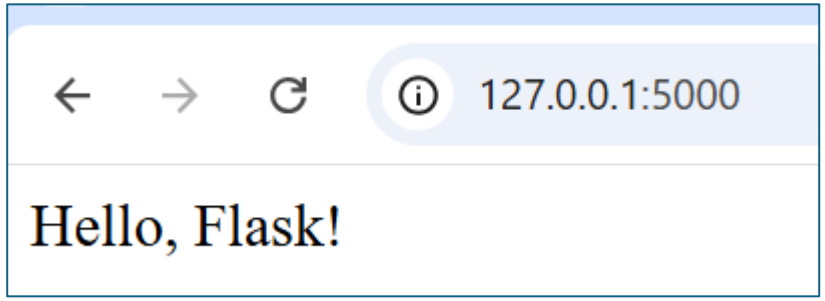
Jinja2 működése: https://jinja.palletsprojects.com/en/stable/

1. példa: Egyszerű Flask alkalmazás

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello, Flask!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Futtatás terminálból: python app.py
Megnyitás: http://127.0.0.1:5000



(alapértelmezett az 5000-es port)

```
 * Serving Flask app 'flask_demo'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.0.64:5000
Press CTRL+C to quit
127.0.0.1 - - [11/Nov/2025 22:47:14] "GET / HTTP/1.1" 200 -
```

A Flask mappaszerkezete

```
Udv_app/
│
├── app.py                  # Flask alkalmazás belépési pontja.
├── templates/              # HTML sablonok (pl. index.html).
│   └── index.html          # Kezdőlap sablonja.
├── static/                 # Statikus fájlok (CSS, JS, képek).
│   ├── css/                # CSS fájlok.
│   ├── js/                 # JavaScript fájlok.
│   └── images/             # Képek.
└── requirements.txt        # Függőségek listája (pl. Flask).
```

opcionális

- **app.py** a Flask alkalmazás belépési pontja. Itt definiáljuk az útvonalakat és az alkalmazás logikáját.
- **templates** mappa tartalmazza az összes HTML sablont. A Flask automatikusan itt keresi a sablonokat.
- **static** mappa tartalmazza a sablonokban hivatkozott statikus fájlokat, pl CSS-t, JavaScriptet és képeket.
- **requirements.txt** fájl tartalmazza a projekt által használt Python csomagokat, amelyeket a pip install -r requirements.txt paranccsal telepíthetünk.

2. példa: Renderelés HTML sablonba

app.py

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def index():
   user = {"name": "Hallgató"}
   return render_template("index.html", user=user)

if __name__ == "__main__":
   app.run(debug=True)
```

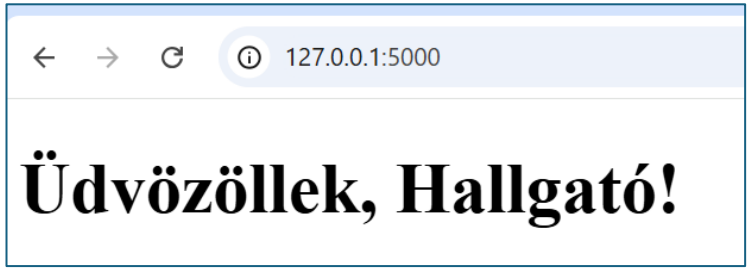
index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Üdvözlet</title>
</head>
<body>
   <h1>Üdvözöllek, {{ user.name }}!</h1>
</body>
</html>
```

- **app = Flask(__name__)** a Flask alkalmazás példányának létrehozása. A __name__ adja meg az alkalmazás gyökérkönyvtárának helyét, ahol a Flask keresi a statikus fájlokat és sablonokat.

- **if __name__ == "__main__":** a kód csak akkor fut, ha a fájlt közvetlenül futtatják (python app.py), és nem akkor, ha egy másik modul importálja.

- **app.run(debug=True)** elindítja a Flask beépített fejlesztői szerverét. A debug=True bekapcsolja a hibakeresési módot. Ez automatikusan újraindítja az alkalmazást kódváltozás esetén, és részletes hiba-információkat jelenít meg a böngészőben.



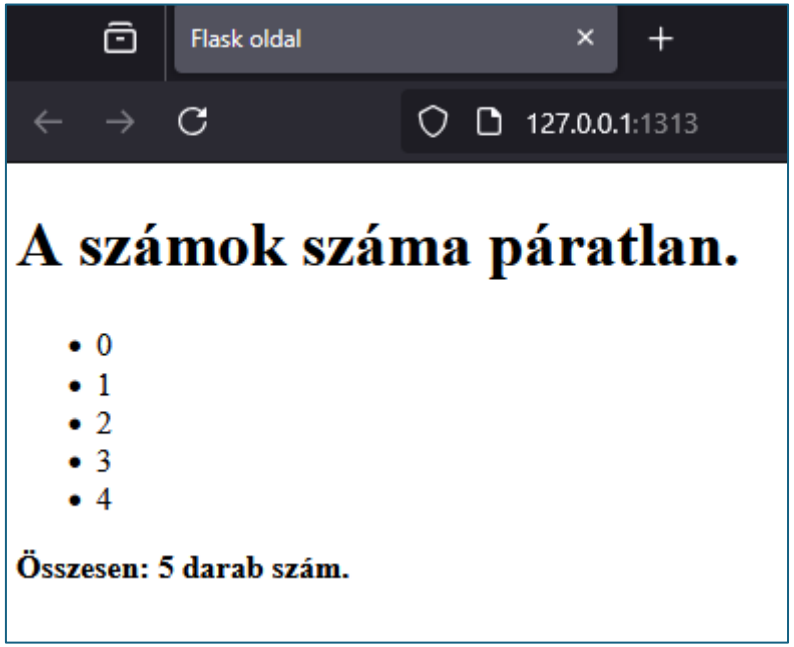
3. példa: flask_test.py

```
from flask import Flask, render_template

app = Flask("flaskpelda")
#futtatás: flask run, vagy python flask_test.py
#fontos a helyes mappaszerkezet!

@app.get("/") #alap oldalra (index.html) vonatkozó kód
def root():
    flask_title = "Flask oldal"
    lst = [i for i in range(5)]
    #ezeket a változókat fogja átadni a HTML oldal számára!
    return render_template("index.html",
                           page_name = flask_title,
                           nums = lst
                           ) #a kulccsal lehet hivatkozni ott rájuk!

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=1313)
```

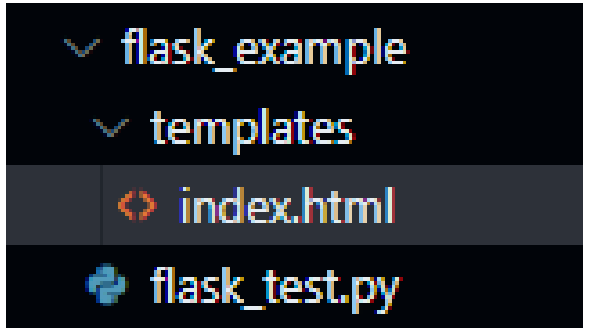
Flask példa – Jinja2 template

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
        …
    <title>{{page_name}}</title>
</head>
<body>
    {% if (nums | length) % 2 == 0 %}
        <h1>A számok száma páros.</h1>
    {% else %}
        <h1>A számok száma páratlan.</h1>
    {% endif %}
    <ul>
        {% for num in nums %}
            <li>{{ num }}</li>
        {% endfor %}
    </ul>
    <b>Összesen: {{(nums | length)}} darab szám.</b>
</body>
</html>
```



Fontos a helyes mappaszerkezet!



Django

Magas szintű, full-stack Python webfejlesztési keretrendszer.

Nagyobb webalkalmazások készítésére használatos.

Összetettebb – kell ismerni hozzá a szerver- és kliensoldali webfejlesztési technikákat.

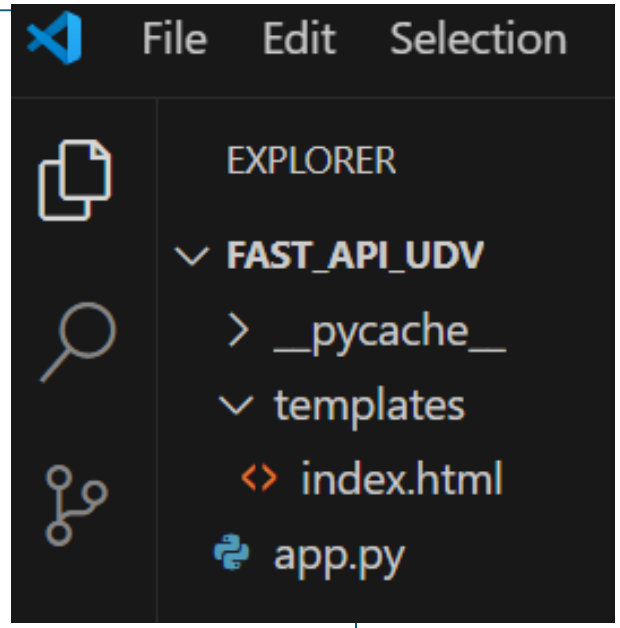
Flask is és Django is összekapcsolható FastAPI-val.

Bővebben Django-ról: https://www.djangoproject.com/

Fast API és Jinja2

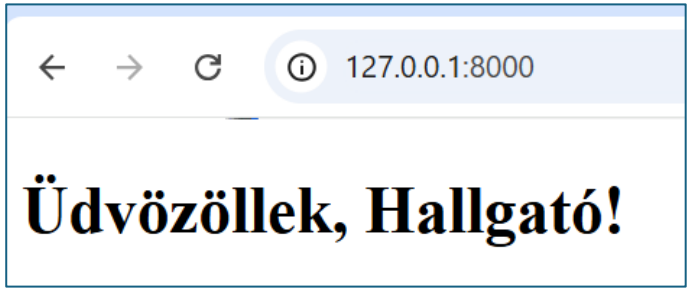
app.py

```
from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles

app = FastAPI()

# Jinja2 sablonok elérési útja
templates = Jinja2Templates(directory="templates")

@app.get("/", response_class=HTMLResponse)
async def index(request: Request):
    user = {"name": "Hallgató"}
    return templates.TemplateResponse("index.html", {"request": request, "user": user})
```



index.html

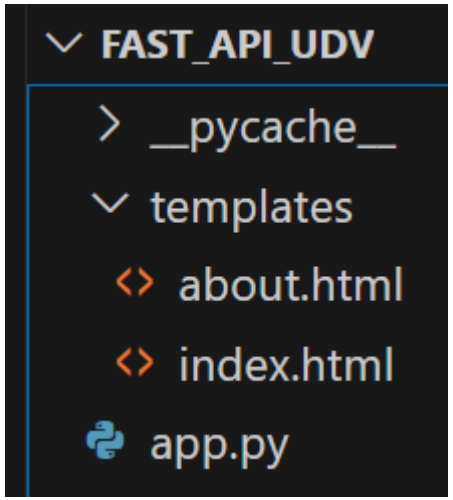
```
<!DOCTYPE html>
<html>
<head>
    <title>Üdvözlet</title>
</head>
<body>
    <h2>Üdvözöllek, {{ user.name }}!</h2>
</body>
</html>
```



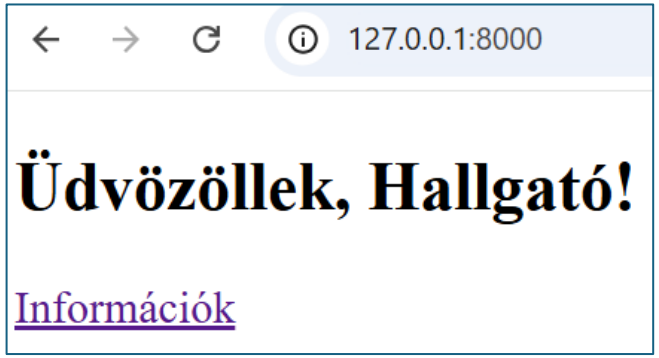
app.py kiegészítése további endpointtal:

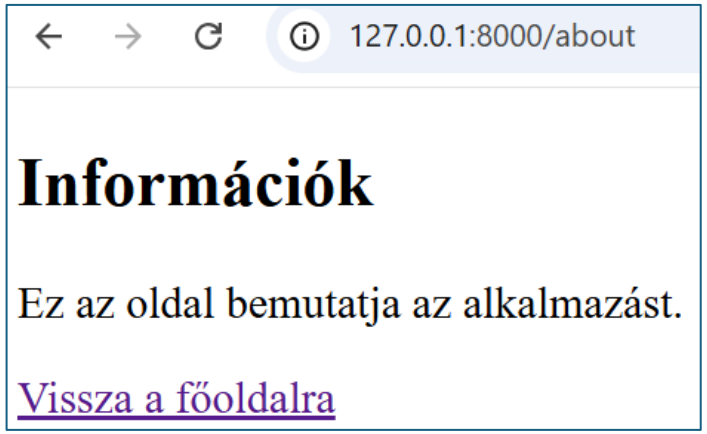
```
@app.get("/about", response_class=HTMLResponse)
async def about(request: Request):
    info = {"title": "Információk", "content": "Ez az oldal bemutatja az alkalmazást."}
    return templates.TemplateResponse("about.html", {"request": request, "info": info})
```

about.html:




```
<!DOCTYPE html>
<html>
<head>
  <title>{{ info.title }}</title>
</head>
<body>
  <h2>{{ info.title }}</h2>
  <p>{{ info.content }}</p>
  <p><a href="/">Vissza a főoldalra</a></p>
</body>
</html>
```





index.html kiegészítése az /about-ra mutató linkkel:

```
<p><a href="/about">Információk</a></p>
```

Egyszerű API létrehozása Flask és Jinja2 segítségével

1. példa: Felhasználói lista megjelenítése

app.py

```
from flask import Flask, render_template, jsonify
app = Flask(__name__)
users = [
  {"id": 1, "name": "Anna"},
  {"id": 2, "name": "Béla"}
]
@app.route("/api/users")      # JSON API végpont
def get_users():
  return jsonify(users)
@app.route("/users")          # HTML oldal megjelenítése
def user_page():
  return render_template("users.html", users=users)
```

Mappa szerkezet:

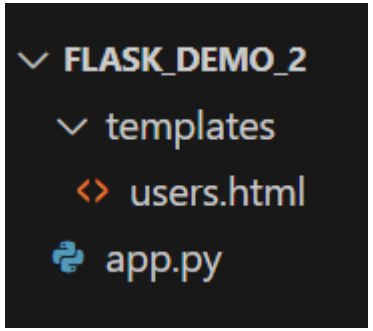
```
flask_demo1/
|— app.py
|— templates/
|   └— users.html
```

users.html részlete:

```
<h1>Felhasználók</h1>
<ul>
 {% for user in users %}
  <li>{{ user.name }}</li>
 {% endfor %}
</ul>
```

2. példa: egyszerű Flask alkalmazás készítése:

- egy végpont, JSON formátumban ad vissza adatokat;
- egy HTML oldal megjelenítése Jinja2 sablonnal;
- alap Flask szerkezet.



app.py

```
from flask import Flask, jsonify, render_template
app = Flask(__name__)
users = [                                                    # Teszt adatok
   {"id": 1, "name": "Anna", "email": "anna@example.com"},
   {"id": 2, "name": "Béla", "email": "bela@example.com"},
   {"id": 3, "name": "Cecil", "email": "cecil@example.com"}
]
@app.route("/api/users")                                     # JSON API végpont
def get_users():
   return jsonify(users)
 @app.route("/users")                                        # HTML oldal megjelenítése
def user_list():
   return render_template("users.html", users=users)
 @app.route("/")                                             # Főoldal
def home():
   return "<h2>Üdv a Flask alkalmazásban!</h2><p>Menj a /users vagy /api/users oldalra.</p>"
if __name__ == "__main__":
   app.run(debug=True)
```

users.html

```
<!DOCTYPE html>
<html lang="hu">
<head>
  <meta charset="UTF-8">
  <title>Felhasználók listája</title>
</head>
<body>
  <h2>Felhasználók</h2>
  <ul>
    {% for user in users %}
      <li>{{ user.name }} – {{ user.email }}</li>
    {% endfor %}
  </ul>
</body>
</html>
```

Az alkalmazás futtatása:

1. Egy virtuális környezet létrehozása (opcionális, de ajánlott):

   python -m venv venv

   venv\Scripts\activate

2. Az alkalmazás futtatása terminálból: **python app.py**

3. Megnyitás böngészőben:
   - http://127.0.0.1:5000 – főoldal
   - http://127.0.0.1:5000/users – HTML oldal
   - http://127.0.0.1:5000/api/users – JSON API végpont

3. példa: Egy nyilvántartó alkalmazás létrehozása Flask-ben:

- Oktatókat és kurzusokat kezel
- Adatokat egy data.json fájlban tárol
- Jinja2 sablonokat használ dinamikus HTML oldalakhoz

Adatmodellek:

- **Kurzus** (név, leírás)
- **Oktató** (név, e-mail)
- **Hallgató** (név, e-mail)
- **Kurzus-Hallgató kapcsolat** (melyik hallgató melyik kurzuson van)

Funkciók:

- Új kurzus/oktató/hallgató felvétele
- Módosítási lehetőség minden entitáshoz
- Törlés
- Hallgatók hozzárendelése kurzusokhoz
- Szűrés: például adott hallgató kurzusainak listázása

Projektstruktúra:

```
course_app/
├── app.py
├── data.json
├── templates/
│   ├── layout.html
│   ├── index.html
│   ├── courses.html
│   ├── add_course.html
│   └── add_instructor.html
```

data.json – kezdeti adatstruktúra

```
{
  "instructors": [],
  "courses": []
}
```

Összetevők:

- **Flask** – szerveroldali keretrendszer
- **Flask-WTF** – űrlapkezeléshez
- **Jinja2** – HTML oldalak rendereléséhez
- **JSON** adatok tárolása
- **Bootstrap** – a megjelenítés stílusához

Jinja2 a HTML oldalba illeszti be a Pythonból jövő adatokat:

```
<!-- courses.html -->
<h1>Kurzusok listája</h1>
<ul>
 {% for course in courses %}
   <li>{{ course.name }} - {{ course.instructor.name }}</li>
 {% endfor %}
</ul>
```

A courses változót a Flask küldi a sablonnak:

```
@app.route("/courses")
def course_list():
   courses = Course.query.all()
   return render_template("courses.html", courses=courses)
```

app.py

```
from flask import Flask, render_template, request, redirect, url_for
import json
import os

app = Flask(__name__)
DATA_FILE = "data.json"

def load_data():
   if not os.path.exists(DATA_FILE):
      with open(DATA_FILE, "w") as f:
         json.dump({"instructors": [], "courses": []}, f)
   with open(DATA_FILE, "r") as f:
      return json.load(f)

def save_data(data):
   with open(DATA_FILE, "w") as f:
      json.dump(data, f, indent=4)

@app.route("/")
def index():
   return render_template("index.html")
```

- app létrehozása
- JSON fájl elérési útvonala
- Fájlkezelő metódusok
- Kezdőlap renderelése

app.py folytatása

```
@app.route("/courses")
def courses():
    data = load_data()
    return render_template("courses.html", courses=data["courses"])

@app.route("/add-course", methods=["GET", "POST"])
def add_course():
    data = load_data()
    if request.method == "POST":
        name = request.form["name"]
        description = request.form["description"]
        instructor_id = int(request.form["instructor"])
        instructor = next((i for i in data["instructors"] if i["id"] == instructor_id), None)
        new_course = {
            "id": len(data["courses"]) + 1,
            "name": name,
            "description": description,
            "instructor": instructor
        }
        data["courses"].append(new_course)
        save_data(data)
        return redirect(url_for("courses"))
    return render_template("add_course.html", instructors=data["instructors"])
```

- **Kurzusok listázása**

- **Kurzus hozzáadása:**
  - attribútumok bekérése:
    request.form-mal
  - new_course dict:
    létrehozás,
    hozzáfűzés, mentés
- redirect és url_for:
  böngésző átirányítása
  a kurzusok listázására

- Sablon renderelése,
  oktatók listájának átadása

app.py folytatása

```
@app.route("/add-instructor", methods=["GET", "POST"])
def add_instructor():
    data = load_data()
    if request.method == "POST":
        name = request.form["name"]
        email = request.form["email"]
        new_instructor = {
            "id": len(data["instructors"]) + 1,
            "name": name,
            "email": email
        }
        data["instructors"].append(new_instructor)
        save_data(data)
        return redirect(url_for("index"))
    return render_template("add_instructor.html")

if __name__ == "__main__":
    app.run(debug=True)
```

- **Oktató hozzáadása**
  - attribútumok bekérése:
    request.form-mal
  - new_instructor dict:
    létrehozás,
    hozzáfűzés,
    mentés
- Redirect, url_for: **böngésző átirányítása** az alapoldalra
- Sablon renderelése, oktatók listájának átadása
- **Futtatás**

Sablonfájlok:

layout.html – a főmenü oldala

```
<!DOCTYPE html>
<html lang="hu">
<head>
  <meta charset="UTF-8">
  <title>Kurzus Nyilvántartó</title>
</head>
<body>
  <h1>Kurzus Nyilvántartó</h1>
  <nav>
    <a href="{{ url_for('index') }}">Főoldal</a> |
    <a href="{{ url_for('courses') }}">Kurzusok</a> |
    <a href="{{ url_for('add_course') }}">Új kurzus</a> |
    <a href="{{ url_for('add_instructor') }}">Új oktató</a>
  </nav>
  <hr>
  {% block content %}{% endblock %}
</body>
</html>
```

- nav bar: információs sáv a menüpontok linkjeivel
- A content blockba kerülnek kitöltésre az egyes menüpontok html kódjai a kiterjesztések szerint.

index.html

```
{% extends "layout.html" %}
{% block content %}
<p>Üdvözöllek a Kurzus Nyilvántartó alkalmazásban!</p>
{% endblock %}
```

courses.html: kurzusok listázása

```
{% extends "layout.html" %}
{% block content %}
<h2>Kurzusok listája</h2>
<ul>
  {% for course in courses %}
    <li><strong>{{ course.name }}</strong> – {{ course.description }}
(Oktató: {{ course.instructor.name }})</li>
  {% else %}
    <li>Nincs elérhető kurzus.</li>
  {% endfor %}
</ul>
{% endblock %}
```

- Alap a layout.html oldal
- Ez a html kerül a content blokk helyére a layout-ban

add_course.html: űrlap (form) megjelenítése

```
{% extends "layout.html" %}
{% block content %}
<h2>Új kurzus felvétele</h2>
<form method="post">
  <label>Kurzus neve:</label><br>
  <input type="text" name="name" required><br>
  <label>Leírás:</label><br>
  <input type="text" name="description"><br>
  <label>Oktató:</label><br>
  <select name="instructor" required>
    {% for instructor in instructors %}
      <option value="{{ instructor.id }}">{{ instructor.name }}</option>
    {% endfor %}
  </select><br><br>
  <button type="submit">Mentés</button>
</form>
{% endblock %}
```

- Alap a layout.html oldal
- Ez a html kerül a content blokk helyére a layout-ban
- Az űrlap POST-ot használ
- 2 szövegmező megjelenítése
- Kötelező kitöltés: required
- Legördülő menü: select
- "submit" gomb: az űrlap elküldése

add_instructor.html: űrlap megjelenítése

```
{% extends "layout.html" %}
{% block content %}
<h2>Új oktató felvétele</h2>
<form method="post">
  <label>Név:</label><br>
  <input type="text" name="name" required><br>
  <label>Email:</label><br>
  <input type="email" name="email"><br><br>
  <button type="submit">Mentés</button>
</form>
{% endblock %}
```

- Alap a layout.html oldal
- Ez a html kerül a content blokk helyére a layout-ban
- Az űrlap POST-ot használ
- 1 szövegmező megjelenítése, kötelező kitöltés: required
- 1 email mező megjelenítése
- "submit" gomb: az űrlap elküldése

Indítás:

```
cd course_app
python app.py
```

Lépésenként kibővítve az alkalmazást:

1. Hallgatók hozzáadása
2. Hallgatók hozzárendelése kurzusokhoz
3. Szűrés: egy hallgató kurzusainak megjelenítése
4. Törlés és módosítás lehetőségek oktatóra, kurzusra, hallgatóra

Frissített projektstruktúra:

```
course_app/
├── app.py
├── data.json
├── templates/
│   ├── layout.html
│   ├── index.html
│   ├── courses.html
│   ├── add_course.html
│   ├── add_instructor.html
│   ├── instructors.html
│   ├── students.html
│   ├── add_student.html
│   ├── enroll_student.html
│   └── student_courses.html
```

Hallgatók hozzáadása

1. Bővítjük a data.json fájlt a hallgatókkal és kapcsolatokkal:

```
{
  "instructors": [],
  "courses": [],
  "students": [],
  "enrollments": []
}
```

2. app.py – új route-ok hozzáadása:

```
@app.route("/students")
def students():
   data = load_data()
   return render_template("students.html",
students=data["students"])

@app.route("/add-student", methods=["GET", "POST"])
def add_student():
   data = load_data()
   if request.method == "POST":
      name = request.form["name"]
      email = request.form["email"]
      new_student = {
         "id": len(data["students"]) + 1,
         "name": name,
         "email": email
      }
      data["students"].append(new_student)
      save_data(data)
      return redirect(url_for("students"))
   return render_template("add_student.html")
```

3. Menüfrissítés (layout.html)

A hallgatói oldalt egészítsük ki ezzel, hogy a hallgatók elérhetőek legyenek a menüből:

```
<a href="{{ url_for('students') }}">Hallgatók</a> |
<a href="{{ url_for('add_student') }}">Új hallgató</a> |
```

4. A students.html sablon létrehozása

templates/students.html

```
{% extends "layout.html" %}
{% block content %}
<h2>Hallgatók listája</h2>
<ul>
  {% for student in students %}
    <li>{{ student.name }} – {{ student.email }}</li>
  {% else %}
    <li>Nincs elérhető hallgató.</li>
  {% endfor %}
</ul>
{% endblock %}
```

5. add_student.html sablon létrehozása

templates/add_student.html

```
{% extends "layout.html" %}
{% block content %}
<h2>Új hallgató felvétele</h2>
<form method="post">
  <label>Név:</label><br>
  <input type="text" name="name" required><br>
  <label>Email:</label><br>
  <input type="email" name="email" required><br><br>
  <button type="submit">Mentés</button>
</form>
{% endblock %}
```

Tesztelés

1. Futtassuk az appot: python app.py
2. Nyissuk meg:
   - http://127.0.0.1:5000/students – hallgatók listája
   - http://127.0.0.1:5000/add-student – új hallgató felvétele

A **Bootstrap** egy nyílt forráskódú **front-end keretrendszer**, amelyet a weboldalak és webalkalmazások gyors és egyszerű fejlesztésére használnak. A Bootstrap HTML, CSS és JavaScript alapú eszközöket biztosít.

Fő jellemzők:

1. **Reszponzív dizájn**: A Bootstrap beépített rácsrendszert biztosít, amely lehetővé teszi, hogy a weboldalak különböző képernyőméretekhez igazodjanak.
2. **Előre definiált stílusok**: Tartalmaz előre elkészített CSS osztályokat gombokhoz, űrlapokhoz, navigációs sávokhoz, táblázatokhoz, kártyákhoz és más UI-elemekhez.
3. **Komponensek**: Számos újrahasználható UI-komponenst kínál, pl. modális ablakokat, legördülő menüket, figyelmeztetéseket, jelvényeket.
4. **JavaScript funkciók**: Tartalmaz JavaScript alapú interaktív elemeket, pl. diavetítéseket (carousel), legördülő menüket és modális ablakokat.
5. **Egyszerű használat**: Könnyen integrálható bármilyen webalkalmazásba, és gyorsan testreszabható a saját igények szerint.

A Bootsrap használata a Flask alkalmazásban:

A Bootstrap integrálható a HTML sablonokba. Ehhez a Bootstrap CSS és JavaScript fájljait kell hozzáadni a sablonokhoz.

Példa: HTML sablon Bootstrap integrációval:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flask + Bootstrap</title>
  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
  <div class="container">
    <h1 class="text-center mt-5">Üdvözöllek a Flask alkalmazásban!</h1>
    <button class="btn btn-primary">Kattints ide</button>
  </div>
  <!-- Bootstrap JS -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Köszönöm a figyelmet!

Konzultáció

Minden héten csütörtökön 18:00 – 20:00